

Product Requirements Document

Python UDP Walkie-Talkie

Real-Time Voice Communication System

Document Type	Technical PRD
Project	Computer Networks - UDP Voice System
Version	1.0
Date	February 2026
Technology Stack	Python 3.x, PyAudio, NumPy, Tkinter

Table of Contents

1. Executive Summary
2. Project Overview
3. Learning Objectives
4. System Architecture
5. Technical Requirements
6. Feature Specifications
7. Audio Processing Pipeline
8. Network Protocol Design
9. User Interface Design
10. Implementation Guide
11. Testing & Validation
12. Deployment Instructions

1. Executive Summary

This document specifies the requirements for building a UDP-based push-to-talk voice communication system. The application demonstrates connectionless networking principles, real-time audio streaming, and packet loss handling strategies used in VoIP and streaming applications.

Key Features

- Push-to-talk voice transmission over UDP
- Real-time audio capture and playback using PyAudio
- Minimal jitter buffer to handle network delays
- Live network debugging overlay showing packet metrics
- Sequence numbering and packet loss estimation

2. Project Overview

2.1 Purpose

The UDP Walkie-Talkie application demonstrates connectionless, fire-and-forget protocol behavior. Unlike TCP's reliability guarantees, UDP prioritizes speed over accuracy—perfect for real-time audio where old data is useless. This project teaches how applications handle packet loss, out-of-order delivery, and latency in streaming scenarios.

2.2 Scope

The application consists of three core components:

Component	Description
Audio Capture	PyAudio module capturing microphone input in 20ms chunks (160 bytes at 8kHz)
UDP Transmission	Datagram socket sending audio frames with sequence numbers to peer device
Audio Playback	Jitter buffer + PyAudio playback managing out-of-order packets and timing

UDP vs TCP for Voice

UDP is ideal for this application because:

- **Low Latency:** No handshake, acknowledgments, or retransmissions
- **Tolerance for Loss:** A few missing audio frames are imperceptible
- **Real-Time Priority:** Old packets are useless—better to drop than delay

3. Learning Objectives

This project reinforces fundamental networking and audio processing concepts:

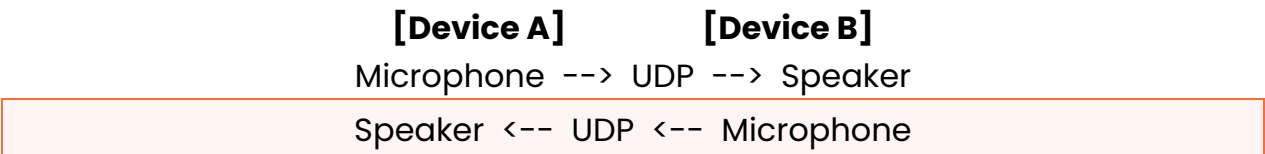
- **UDP Protocol:** Understand connectionless communication and fire-and-forget behavior
- **Datagram Sockets:** Implement UDP sockets for sending and receiving datagrams

- **Audio Streaming:** Capture, encode, transmit, decode, and playback real-time audio
- **Jitter Buffering:** Handle variable network delays and out-of-order packet arrival
- **Packet Loss Detection:** Track sequence numbers to estimate dropped packets
- **Network Metrics:** Calculate throughput, latency, and packet loss rate

4. System Architecture

4.1 High-Level Architecture

The system operates in a peer-to-peer configuration:



4.2 Audio Pipeline Flow

Stage	Process
1	Capture: PyAudio reads from microphone (8kHz, mono, 16-bit PCM)
2	Chunk: Segment into 20ms frames (160 bytes)
3	Package: Wrap audio data with 8-byte header (sequence number + timestamp)
4	Transmit: Send UDP datagram to peer IP:Port
5	Receive: Peer receives datagram, extracts header
6	Buffer: Add to jitter buffer (50ms capacity, reorder by sequence)
7	Playback: PyAudio writes to speaker, dropping late/duplicate packets

5. Technical Requirements

5.1 Technology Stack

Category	Technology
Language	Python 3.8+
Audio	PyAudio (PortAudio wrapper)
Networking	socket (UDP), struct (binary packing)
UI	Tkinter (Python GUI)
Data Processing	NumPy (for buffer operations)

5.2 Python Dependencies

Install required packages:

```
pip install pyaudio numpy
```

Note on PyAudio Installation

PyAudio may require additional system libraries:

- **Windows:** Download precompiled wheel from Gohlke's repository
- **macOS:** brew install portaudio
- **Linux:** sudo apt-get install portaudio19-dev (Ubuntu/Debian)

6. Feature Specifications

6.1 Core Features

6.1.1 Push-to-Talk Mechanism

Requirements:

- Hold SPACE key or on-screen button to transmit
- Visual indicator showing transmission state (LED-style)
- Audio capture only active during transmission
- Automatic mute when not transmitting to prevent feedback

6.1.2 UDP Audio Streaming

Requirements:

- Send audio frames via UDP datagrams (no connection establishment)
- Receive on separate thread to prevent blocking
- Handle socket timeouts gracefully
- Support configurable remote peer IP and port

6.1.3 Jitter Buffer

Requirements:

- Capacity for 2-3 packets (40-60ms latency)
- Reorder packets based on sequence number
- Drop duplicate packets
- Discard packets that arrive too late
- Synthesize silence for lost packets

6.2 Network Debugging Overlay

Real-time metrics displayed in application window:

Metric	Description
Sequence Number	Current packet sequence (increments with each frame sent)
Packet Loss %	$(\text{Expected} - \text{Received}) / \text{Expected} \times 100$
Bytes/Second	Audio data throughput (excluding headers)
Source/Dest	Sender IP:Port → Receiver IP:Port

Buffer Occupancy	Packets in jitter buffer / Max capacity
Latency Estimate	Network delay (timestamp delta) in milliseconds

7. Audio Processing Pipeline

7.1 Audio Format Specification

Parameter	Value
Sample Rate	8000 Hz (8kHz) - telephone quality
Channels	1 (Mono)
Bit Depth	16-bit (signed integer)
Frame Size	160 bytes (80 samples × 2 bytes/sample)
Frame Duration	20 milliseconds (0.02 seconds)
Throughput	8 KB/sec (160 bytes × 50 frames/sec)

7.2 Capture and Encoding

PyAudio Configuration:

```
import pyaudio FORMAT = pyaudio.paInt16 # 16-bit CHANNELS = 1 #
Mono RATE = 8000 # 8kHz CHUNK = 160 # 160 bytes = 20ms
audio = pyaudio.PyAudio() stream = audio.open( format=FORMAT,
channels=CHANNELS, rate=RATE, input=True,
frames_per_buffer=CHUNK )
```

7.3 Decoding and Playback

Output stream configuration:

```
output_stream = audio.open( format=FORMAT, channels=CHANNELS,
rate=RATE, output=True, frames_per_buffer=CHUNK )
output_stream.write(audio_data) # Play received frame
```

8. Network Protocol Design

8.1 Packet Structure

Each UDP datagram contains an 8-byte header + 160-byte audio payload:

Field	Type	Size	Description
Sequence	uint32	4 bytes	Packet counter
Timestamp	uint32	4 bytes	Milliseconds
Audio Data	bytes	160 bytes	PCM samples

8.2 Packet Encoding

```
import struct import time sequence_number = 0
def create_packet(audio_data):
    global sequence_number
    timestamp = int(time.time() * 1000) # milliseconds
    header = struct.pack("!!I", sequence_number, timestamp)
    packet = header + audio_data
    sequence_number += 1
    return packet
```

8.3 Packet Decoding

```
def parse_packet(packet):
    header = packet[:8]
    audio_data = packet[8:]
    sequence, timestamp = struct.unpack("!!I", header)
    return sequence, timestamp, audio_data
```

9. User Interface Design

9.1 Tkinter Layout

```
+-----+
|  UDP WALKIE-TALKIE  |
+-----+
| Peer IP: [192.168.1.100 ] Port:|
| [5000]  [Connect]      |
+-----+
|                               |
| [PUSH TO TALK BUTTON]      |
| (Hold SPACE or Click)      |
|                               |
| Status:  Idle      |
+-----+
| DEBUG PANEL              |
| Seq: 1234  Loss: 2.3%      |
| Throughput: 8.1 KB/s      |
| Buffer: 2/3  Latency: 35ms  |
+-----+
```

9.2 UI Components

9.2.1 Connection Settings

- **Peer IP Entry:** Text field for entering remote device IP address
- **Port Entry:** Text field for UDP port (default 5000)
- **Connect Button:** Initializes UDP socket and starts receive thread

9.2.2 Push-to-Talk Button

- **Visual States:**
 -  Idle (white) - Not transmitting
 -  Transmitting (red) - Microphone active
 -  Receiving (green) - Playing incoming audio

9.2.3 Debug Panel

Display as monospaced text with live updates every 100ms:

```
SEQ: 00001234 | LOSS: 2.3% THRU: 8.1 KB/s | LAT: 35ms BUFFER: [██████] 2/3 packets
```

10. Implementation Guide

10.1 Project Structure

```
udp_walkie_talkie/  
├── main.py          # Application entry point  
├── audio_handler.py # PyAudio capture/playback  
├── network_handler.py # UDP send/receive  
├── jitter_buffer.py  # Packet reordering logic  
├── metrics.py        # Network statistics  
├── ui.py             # Tkinter interface  
└── requirements.txt  # Dependencies
```

10.2 Core Implementation Steps

Step 1: Audio Capture Thread

Key Logic:

- Continuously read from microphone when PTT active
- Package audio with sequence number and timestamp
- Send via UDP socket
- Update throughput metrics

Step 2: UDP Receive Thread

Key Logic:

- Listen on UDP socket with timeout
- Parse header to extract sequence and timestamp
- Add packet to jitter buffer
- Update packet loss metrics

Step 3: Jitter Buffer Management

Pseudo-code:

```
class JitterBuffer:
    def __init__(self, capacity=3):
        self.buffer = [] # List of (seq, timestamp, audio)
        self.capacity = capacity
        self.last_played_seq = -1
    def add(self, packet):
        # Insert maintaining sorted order by sequence
        self.buffer.append(packet)
        if len(self.buffer) > self.capacity:
            self.buffer.pop(0) # Drop oldest
    def get_next(self):
        if not self.buffer:
            return None # Silence
        packet = self.buffer.pop(0)
        self.last_played_seq = packet[0]
        return packet[2] # Return audio data
```

11. Testing & Validation

11.1 Functional Tests

Test #	Test Case	Expected Result
1	Basic transmission	Speak into Device A, hear on Device B with <100ms delay
2	Bidirectional communication	Both devices can transmit and receive alternately
3	Packet loss handling	Simulate 5% loss using tc/netem - audio still intelligible
4	Out-of-order packets	Jitter buffer reorders, audio plays smoothly
5	Metrics accuracy	Loss rate matches network simulation tool settings
6	PTT responsiveness	Button activates transmission within 50ms of press

11.2 Network Simulation Tests

Using Linux tc (traffic control) to simulate real-world conditions:

```
# Add 50ms latency sudo tc qdisc add dev eth0 root netem delay 50ms #  
Add 10% packet loss sudo tc qdisc change dev eth0 root netem loss 10% #  
Add jitter (±10ms variation) sudo tc qdisc change dev eth0 root netem delay  
50ms 10ms
```

12. Deployment Instructions

12.1 Setup Process

Step 1: Install Dependencies

```
pip install pyaudio numpy
```

Step 2: Configure Network

- Both devices must be on same network or have routable IPs
- Allow UDP port 5000 through firewall
- Find IP addresses using ifconfig (Linux/Mac) or ipconfig (Windows)

Step 3: Run Application

On Device A:

```
python main.py # Enter Device B's IP in UI # Click Connect
```

On Device B:

```
python main.py # Enter Device A's IP in UI # Click Connect
```

12.2 Troubleshooting

Issue	Solution
-------	----------

No audio received	Check firewall rules. Verify IP addresses. Test with ping.
Choppy audio	Increase jitter buffer size. Check network latency. Reduce background traffic.
PyAudio errors	Verify microphone permissions. Check device index with <code>p.get_device_info_by_index()</code>
High packet loss	Switch to wired connection. Move closer to router. Check for interference.

Document End

This PRD provides comprehensive specifications for building a UDP-based voice communication system. Follow the implementation guide to create a working walkie-talkie that demonstrates connectionless networking, real-time audio streaming, and packet loss handling strategies.